Secure Password Database

[Pico CTF '26] Secure Password Database

This a medium difficulty Pico CTF reverse engineering challenge. The name has "password" and "database" so perhaps many passwords are being stored and we need to find one containing the flag?

Enumeration

All we get for this challenge is the binary file, `system.out`. Running some preliminary checks on the file...

```
demo-academy@webshell:~$ file system.out
system.out: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=63224e5a94fa31cb071c82105d8a70ffc806ac0b, for GNU/Linux 3.2.0,
not stripped

demo-academy@webshell:~$ checksec ./system.out
[*] '/home/demo-academy/system.out'
    Arch:       amd64-64-little
    RELRO:      Full RELRO
    Stack:      Canary found
    NX:         NX enabled
    PIE:        PIE enabled
    SHSTK:      Enabled
    IBT:        Enabled
    Stripped:   No
```

Next, I opted to open the binary in Ghidra. Looking at the Symbol Tree pane, there are three methods which garner my interest. These are

1. `main()`
2. `make_secret()`

3. `hash()`

   `make_secret()` takes a long as a parameter and based off of the Function Call Graph screen, is responsible for calling `hash()` which takes a Ghidra's `byte*` as a parameter. The main method has a total of 17 local variables:

```
5   uint uVar1;
6   char *pcVar2;
7   undefined8 uVar3;
8   long in_FS_OFFSET;
9   int local_128;
10  char *local_120;
11  ulong local_118;
12  char *local_110;
13  size_t local_108;
14  ulong local_100;
15  ulong local_f8;
16  FILE *local_f0;
17  undefined1 local_e5 [13];
18  char local_d8 [31];
19  char acStack_b9 [65];
20  char local_78 [104];
21  long local_10;
```

Analysis

The program starts by allocating 90 bytes on the heap and stores the address at `local_110`. I'll call this variable `heap_ptr` because I don't know much about its purpose yet. It then enters a loop that iterates over 13 bytes of memory starting at `heap_ptr[60]`, storing the result of a curious binary operation: `obf_bytes[i] ^ 0b10101010`. Next we get our first message printed to `stdout`: "Please set a password for your account". It uses `fgets()` to store the password in a 65 byte buffer which I will call `password`. The pointer returned by `fgets()` is stored in `pcVar2` and followed by an if-statement that checks if it does NOT equal `(char*)0` AKA `NULL`.

> **Tip**
>
> This process of fetching input and checking to make sure it is valid is repeated after EVERY call to `fgets()` in the program, and works the same way every time. I will ignore it for the rest of my program analysis.

The password the user enters is then copied from the stack to the *beginning* of the 90 bytes referred to by `heap_ptr` (as opposed to the still cryptic series of bytes starting at `heap_ptr[60]`). Next we're asked to input a number for the question: "How many bytes in length is your password?". That's right -- rather than using `strlen()` to get the answer, the program hands *us* the opportunity to answer. The result is initially stored in a 31 byte buffer before being converted to an integer with `atoi()` and storing the result in `uVar1`. I will name this variable `password_len`. The program tells us the number we entered, and proceeds to print out the decimal value of each character in our password **up to** `password_len`. It does this via a loop starting at `heap_ptr[0]`. There is no honesty check here -- if we want to tell the program our password is of a completely different length than it actually is, we *can*. This allows us to peer further into the 90 bytes, and more specifically, at what lies at `heap_ptr[60]`.

Next we are asked to input our account's *hash*. Our input overwrites our password that was being stored on the stack which luckily for us, is still stored in the heap. It is unclear exactly what this hash is but the closest thing that comes to mind is the decimal values of our password that got printed out earlier. Whatever the case, the newline that gets scooped up with our input is replaced by a null terminator. We then convert his hash into a long using `strtoul()` and store the result in `local_100`. I will name this variable `user_in_long`. We use the middle argument to `strtoul()` (which I will name `leftovers`) to see if the conversion failed. If it did, we shut down the program with `__assert_fail`.

If the conversion was from string to long was successful however, we store the result of `make_secret(local_e5)` in `local_f8`. Although not much is known about `make_secret()`, I will change `local_f8` -> `make_secret_result`. But before jumping into `make_secret()` I would like to point out that the next line is an if-statement that checks if `make_secret_result == user_in_long`, and just beyond *that*, `flag.txt` is opened, and the flag is read from the file. Clearly whatever happens in `make_secret()` is crucial to solving this challenge.

```
2  void make_secret(long param_1)
3
4  {
5    long local_10;
6
7    for (local_10 = 0; obf_bytes[local_10] != '\0'; local_10 = local_10 + 1) {
8      *(byte *)(local_10 + param_1) = obf_bytes[local_10] ^ 0b10101010;
9    }
10   *(undefined1 *)(param_1 + 12) = 0;
11   hash(param_1);
12   return;
13 }
```

Things are already strange stepping into `make_secret()`:

1. it says the function is of type void, but we are storing the result of the function into `local_f8`. Well void is specifying that we *aren't* returning anything so how can this work?
2. Ghidra is listing the parameter taken by `make_secret()` as a long, but we call the function with `local_e5` (type unknown). All Ghidra's decompilation window tells us it is at least an array with 13 spots.
   We can see the function sets up one long called `local_10` which gets used as an index variable for a for-loop (I will name this variable `i`) and -- hey! We've seen this before! It stores the result of binary operation: `obf_bytes[i] ^ 0b10101010` in an address pointed to by `param_1`, offset by `i`. We know `param_1` is `local_e5` when the function is called, and it just so happens that both `obf_bytes` and `local_e5` are arrays with 13 spots (recall the for-loop in the beginning iterating 13 times over `obf_bytes`). With this information I will change `local_e5`'s type to a `byte*`. I will also change the name of `local_e5` -> `enc_obf_bytes` because after the loop it is essentially `obf_bytes` with its bytes encrypted in some way. It sets the last byte to a null terminator, and then calls `hash(enc_obf_bytes)`.
   Before jumping into that call however, I want to look at the last lines of `make_secret()`. We see `make_secret()` has a `return` statement but fails to specify anything to return. On one hand, this follows because the function is of type `void`, but what is then stored in `make_secret_result`?

```
long hash(byte *param_1)

{
  byte *local_20;
  long local_10;

  local_10 = 5381;
  local_20 = param_1;
  while( true ) {
    if (*local_20 == 0) break;
    local_10 = (long)(int)(uint)*local_20 + local_10 * 33;
    local_20 = local_20 + 1;
  }
  return local_10;
}
```

`hash()`'s function header says it takes a `byte*`, further confirming that `enc_obf_bytes` should really be of type `byte*` since that's what gets passed to the function. This function has two local variables:

1. `byte *local_20`
2. `long local_10`

   The function sets the `local_20` to point at whichever address `enc_obf_bytes` is looking at and sets `local_10 = 5381`. I will rename these so they are less ugly to look at although still not much is known about them: `local_20 -> pByte` and `local_19 -> mystery_value`.

   The function then enters a seemingly infinite while-loop only breaking when the character `pByte` is looking at is `\0`. That should be fine as, if you recall, in `make_secret()` we explicitly set a null byte at `enc_obf_bytes[12]` meaning the loop should iterate 12 times. If the character is NOT null however, we update `mystery_value` to whatever the current value of `mystery_value` is `* 33 + *pByte`'s ascii integer value (type cast as a long). Finally we increment `pByte` to look at the next character. Whatever is in `mystery_value` by the end is returned. Since `mystery_value` is a `long` and `make_secret_result` is also a long, could `mystery_value` actually be what is stored in `make_secret_result`?

Walkthrough

Okay so it is evident I get the flag by inputting `make_secret_result` which itself is a `long` generated by the values in `enc_obf_bytes`. Well, we saw `enc_obf_bytes` was filled the exact same way `heap_ptr[60]` was -- that is to say by looking at the 13 bytes starting at `heap_ptr[60]`, we can see what `enc_obf_bytes` is. I could do this with a debugger like gdb, but as stated earlier I can say my password has a length of 90 to view all the bytes allocated for `heap_ptr`:

```
demo-academy@webshell:~$ ./system.out
Please set a password for your account:
password
How many bytes in length is your password?
90
You entered: 90
Your successfully stored password:
112 97 115 115 119 111 114 100 10 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 105 85 98 104 56 49 33 106 42 104 110 33 -86 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Enter your hash to access your account!
demo-academy@webshell:~$
exit
```

*Please note that this screenshot does not show ALL of the 90 bytes for the sake of fitting on the page.

As expected we see what the 13 bytes copied to `heap_ptr[60]` represented as integers: `105 85 98 104 56 49 33 106 42 104 110 33 -86`. This is exactly the same as is in `enc_obf_bytes`. This is still not enough however, because `hash()` iterates over these bytes and returns a final value which is what we ACTUALLY want to feed the program. Well, rather than doing the calculations ourselves, I can easily see what value the program returns using gdb. Because PIE is enabled, I'll set a breakpoint at `hash()` and then one at the end of the function to see what value is returned in `rax`:

```
Breakpoint 2, 0x00005cfadc1dd35c in hash ()
(gdb) disas hash
Dump of assembler code for function hash:
   0x00005cfadc1dd309 <+0>:     endbr64
   0x00005cfadc1dd30d <+4>:     push   rbp
   0x00005cfadc1dd30e <+5>:     mov    rbp,rsp
   0x00005cfadc1dd311 <+8>:     mov    QWORD PTR [rbp-0x18],rdi
   0x00005cfadc1dd315 <+12>:    mov    QWORD PTR [rbp-0x8],0x1505
   0x00005cfadc1dd31d <+20>:    jmp    0x5cfadc1dd33d <hash+52>
   0x00005cfadc1dd31f <+22>:    mov    rax,QWORD PTR [rbp-0x8]
   0x00005cfadc1dd323 <+26>:    shl    rax,0x5
   0x00005cfadc1dd327 <+30>:    mov    rdx,rax
   0x00005cfadc1dd32a <+33>:    mov    rax,QWORD PTR [rbp-0x8]
   0x00005cfadc1dd32e <+37>:    add    rdx,rax
   0x00005cfadc1dd331 <+40>:    mov    eax,DWORD PTR [rbp-0xc]
   0x00005cfadc1dd334 <+43>:    cdqe
   0x00005cfadc1dd336 <+45>:    add    rax,rdx
   0x00005cfadc1dd339 <+48>:    mov    QWORD PTR [rbp-0x8],rax
   0x00005cfadc1dd33d <+52>:    mov    rax,QWORD PTR [rbp-0x18]
   0x00005cfadc1dd341 <+56>:    lea    rdx,[rax+0x1]
   0x00005cfadc1dd345 <+60>:    mov    QWORD PTR [rbp-0x18],rdx
   0x00005cfadc1dd349 <+64>:    movzx  eax,BYTE PTR [rax]
   0x00005cfadc1dd34c <+67>:    movzx  eax,al
   0x00005cfadc1dd34f <+70>:    mov    DWORD PTR [rbp-0xc],eax
   0x00005cfadc1dd352 <+73>:    cmp    DWORD PTR [rbp-0xc],0x0
   0x00005cfadc1dd356 <+77>:    jne    0x5cfadc1dd31f <hash+22>
   0x00005cfadc1dd358 <+79>:    mov    rax,QWORD PTR [rbp-0x8]
=> 0x00005cfadc1dd35c <+83>:    pop    rbp
   0x00005cfadc1dd35d <+84>:    ret
End of assembler dump.
(gdb) i r rax
rax            0xd3770d6251b31be2  -3209081493549540382
(gdb)
```

And just like that I know the value of `make_secret_result` to be `! I can see that since` make_secret() `calls` hash() `which returns its result in` rax `, it is really` hash() `that has the final say over what is copied to` make_secret_result (rbp-0xf0)`:

```
call make_secret
mov QWORD PTR [rbp-0xf0], rax
```

Now I just have to provide `-3209081493549540382` to the program and I should get the flag.

Solution

I will write a quick python script in the command line and pipe it into the program. It should be noted `-3209081493549540382` **needs** to be provided in base 10 because the `strtoul()` call in the program specifies to parse them in base 10:
`user_in_long = strtoul(password + 1,&leftovers,10)`
My python script of course needs to answer the 2 prompts that come before the hash question but just know only the `-3209081493549540382` needs to be the same.

```
python3 -c "print('password'); print('90'); print('-3209081493549540382')"
```

And by piping that into the program we should receive the flag!

```
picoCTF{d0nt_trust_us3rs}
```